

1-Finding Duplicates- $O(n^2)$ Time Complexity, $O(1)$ Space Complexity

Find Duplicate in Array.

Given a read only array of n integers between 1 and n , find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

Answer:(penalty regime: 0 %)

```

1  #include <stdio.h>
2
3  int main() {
4      int n;
5      scanf("%d", &n);
6
7      int a[n];
8
9      for (int i = 0; i < n; i++) {
10         scanf("%d", &a[i]);
11     }
12
13     for (int i = 0; i < n; i++) {
14         for (int j = i + 1; j < n; j++) {
15             if (a[i] == a[j]) {
16                 printf("%d", a[i]);
17                 return 0;
18             }
19         }
20     }
21
22     return 0;
23 }

```

	Input	Expected	Got	
✓	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✓
✓	5 1 2 3 4 4	4	4	✓
✓	5 1 1 2 3 4	1	1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

2-Finding Duplicates- $O(n)$ Time Complexity, $O(1)$ Space Complexity

Find Duplicate in Array.

Given a read only array of n integers between 1 and n , find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

Answer:(penalty regime: 0 %)

```

1  #include <stdio.h>
2
3  int main() {
4      int n;
5      scanf("%d", &n);
6
7      int a[n];
8
9      for (int i = 0; i < n; i++) {
10         scanf("%d", &a[i]);
11     }
12
13     for (int i = 0; i < n; i++) {
14         for (int j = i + 1; j < n; j++) {
15             if (a[i] == a[j]) {
16                 printf("%d", a[i]);
17                 return 0;
18             }
19         }
20     }
21
22     return 0;
23 }

```

	Input	Expected	Got	
✓	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✓
✓	5 1 2 3 4 4	4	4	✓
✓	5 1 1 2 3 4	1	1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

3-Print Intersection of 2 sorted arrays- $O(m*n)$ Time Complexity, $O(1)$ Space Complexity

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N_1 , followed by N_1 integers of the first array
2. Line 2 contains N_2 , followed by N_2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57	10 57

246	
-----	--

Answer:(penalty regime: 0 %)

```
1  #include <stdio.h>
2  |
3  ▾ int main() {
4      int T;
5      scanf("%d", &T);
6
7  ▾  while (T--) {
8      int n1, n2;
9      scanf("%d", &n1);
10
11     int a[n1];
12
13  ▾     for (int i = 0; i < n1; i++) {
14         scanf("%d", &a[i]);
15     }
16
17     scanf("%d", &n2);
18
19     int b[n2];
20
21  ▾     for (int i = 0; i < n2; i++) {
22         scanf("%d", &b[i]);
23     }
24
25     int i = 0, j = 0;
26     int first = 1;
27
28  ▾     while (i < n1 && j < n2) {
29  ▾         if (a[i] == b[j]) {
30             if (!first)
31                 printf(" ");
32
33             printf("%d", a[i]);
34             first = 0;
```

```

36         i++;
37         j++;
38     }
39     else if (a[i] < b[j]) {
40         i++;
41     }
42     else {
43         j++;
44     }
45 }
46
47     printf("\n");
48 }
49
50     return 0;
51 }

```

	Input	Expected	Got	
✓	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✓
✓	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✓

Passed all tests! ✓

4-Print Intersection of 2 sorted arrays- $O(m+n)$ Time Complexity, $O(1)$ Space Complexity

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N_1 , followed by N_1 integers of the first array
2. Line 2 contains N_2 , followed by N_2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57	10 57

Answer:(penalty regime: 0 %)

```
1  #include <stdio.h>
2
3  int main() {
4      int T;
5      scanf("%d", &T);
6
7      while (T--) {
8          int n1, n2;
9          scanf("%d", &n1);
10
11         int a[n1];
12
13         for (int i = 0; i < n1; i++) {
14             scanf("%d", &a[i]);
15         }
16
17         scanf("%d", &n2);
18
19         int b[n2];
20
21         for (int i = 0; i < n2; i++) {
22             scanf("%d", &b[i]);
23         }
24
25         int i = 0, j = 0;
26         int first = 1;
27
28         while (i < n1 && j < n2) {
29             if (a[i] == b[j]) {
30                 if (!first)
31                     printf(" ");
32
33                 printf("%d", a[i]);
34                 first = 0;
35             }
```

```

35
36         i++;
37         j++;
38     }
39     else if (a[i] < b[j]) {
40         i++;
41     }
42     else {
43         j++;
44     }
45 }
46
47     printf("\n");
48 }
49
50     return 0;
51 }

```

	Input	Expected	Got	
✓	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✓
✓	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✓

Passed all tests! ✓

5-Pair with Difference- $O(n^2)$ Time Complexity, $O(1)$ Space Complexity

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that $A[j] - A[i] = k$, $i \neq j$.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as $5 - 1 = 4$

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

Answer:(penalty regime: 0 %)

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int n, k;
6      scanf("%d", &n);
7
8      int a[n];
9
10     for (int i = 0; i < n; i++)
11         scanf("%d", &a[i]);
12
13     scanf("%d", &k);
14
15     int i = 0, j = 1;
16
17     while (i < n && j < n)
18     {
19         if (i == j)
20         {
21             j++;
22         }
23         else if (a[j] - a[i] == k)
24         {
25             printf("1");
26             return 0;
27         }
28         else if (a[j] - a[i] < k)
29         {
30             j++;
31         }
32         else
```

```

32         else
33         {
34             i++;
35         }
36     }
37
38     printf("0");
39
40     return 0;
41 }

```

	Input	Expected	Got	
✓	3 1 3 5 4	1	1	✓
✓	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✓
✓	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✓
✓	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✓

Passed all tests! ✓

6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that $A[j] - A[i] = k$, $i \neq j$.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as $5 - 1 = 4$

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

Answer:(penalty regime: 0 %)

```
1  #include <stdio.h>
2
3  int main() {
4      int n, k;
5      scanf("%d", &n);
6
7      int a[n];
8
9      for (int i = 0; i < n; i++) {
10         scanf("%d", &a[i]);
11     }
12
13     scanf("%d", &k);
14
15     int i = 0, j = 1;
16
17     while (i < n && j < n) {
18         if (i == j) {
19             j++;
20         }
21         else if (a[j] - a[i] == k) {
22             printf("1");
23             return 0;
24         }
25         else if (a[j] - a[i] < k) {
26             j++;
27         }
28         else {
29             i++;
30         }
31     }
32
33     printf("0");
34
35     return 0;
36 }
```

	Input	Expected	Got	
✓	3 1 3 5 4	1	1	✓
✓	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✓
✓	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✓
✓	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.